

AI技术调研与场景分析

项目名称：考拉在线考试系统

姓名：杨策华

学号：2023141461207

1. 项目介绍与文档说明

根据作业要求，本报告重点关注 2024—2026 年 AI 在软件工程领域的应用趋势，覆盖需求分析、系统设计、编码实现、测试验证、运维监控等生命周期环节。分析不以“AI 替代开发人员”为目标，而是以“人机协同的软件过程优化”为原则，判断 AI 是否能够在本项目现有流程中降低重复劳动、提高文档与代码质量、增强测试覆盖、辅助缺陷定位，并在小规模学生团队中保持可落地性。

本项目为“考拉在线考试系统”，主要面向小学至高中阶段的教师、学生及教育管理者，提供考试、作业、题库、试卷、错题本、消息、成绩分析和用户管理等功能。系统目标是减少传统纸质考试与作业管理中的人工分发、批改和统计负担，提升教学管理效率和学生自主学习效果。

2. 2024—2026 年 AI 在软件工程中的应用趋势调研

2.1 总体趋势：从单点代码补全走向全生命周期协作

2024 年以后，AI 在软件工程中的应用已经不再局限于代码自动补全，而是逐渐覆盖需求、设计、编码、测试、评审、运维等完整生命周期[1][2][4]。GitHub Copilot、Cursor、JetBrains AI、ChatGPT、Claude、Gemini 等工具将大语言模型接入 IDE、代码仓库、命令行、Pull Request 和文档系统，使开发人员可以围绕自然语言提示、项目上下文、代码片段和运行日志开展交互式开发[4][5]。

DORA 2024 与 2025 年关于 AI 辅助软件开发的报告都强调：AI 对个人生产率、开发流畅度和满意度具有积极作用，但它不是单纯的“效率加速器”；DORA 2025 进一步指出，AI 更像是组织能力的“放大器”，会放大团队原有优势与短板[1][2]。如果团队缺少良好的测试、评审和小批量交付机制，AI 可能会放大已有问题。因此，在本项目中引入 AI 时，应强调“AI 生成 + 人工审核 + 度量反馈”的闭环，而不能直接把 AI 输出作为最终交付物。

Stack Overflow 2025 开发者调查显示，越来越多开发者已经在软件开发中使用或计划使用 AI 工具，但开发者对 AI 输出的准确性仍保持谨慎；该调查页面显示，对 AI 工具准确性表示不信任的开发者比例高于表示信任的比例[3]。这说明 AI 在软件工程中已经具备现实普及基础，但仍需要人类开发者进行判断、验证和责任承担。对于考拉在线考试系统这样的课程项目，AI 的价值主要体现在减少重复文档编写、生成候选测试用例、辅助接口与 SQL 检查、帮助快速定位低级错误，而不是替代架构决策和最终质量验收。

2.2 需求分析阶段：AI 辅助需求捕获、整理与一致性检查

在需求分析阶段，生成式 AI 可以根据访谈记录、用户故事、竞品分析、功能清单和业务背景，生成需求条目、用例描述、验收标准、异常流程和需求优先级建议[7][11]。对于教育考试系统而言，教师、学生和管理员的使用场景较多，人工整理时容易遗漏边界情况，例如试卷发布后是否允许修改、考试提交失败如何处理、错题删除后是否影响历史记录、管理员禁用账号后学生是否还能查看历史成绩等。AI 可以在需求文档初稿完成后，从一致性、完整性、可测试性角度进行检查，列出潜在矛盾和遗漏点。

近年的生成式 AI 需求工程研究表明，LLM 可以用于需求分类、需求摘要、需求冲突发现、验收标准生成和用户故事补全[7][11]。但 AI 对业务上下文和隐含约束的理解仍有限，容易产生看似合理但并不存在于项目范围内的功能。因此，AI 在本项目需求阶段的合适定位是“辅助分析员”，用于生成候选需求、检查遗漏和优化表述，最终仍由团队依据课程目标和项目边界确认。

2.3 系统设计阶段：AI 辅助架构解释、UML/ER 草图生成与设计评审

系统设计阶段的 AI 应用主要体现在三方面：第一，根据需求文档生成候选的模块划分、类图、顺序图和活动图；第二，根据数据库表结构与业务流程检查 ER 模型、字段命名、外键关系、权限控制和冗余问题；第三，结合已选技术栈给出架构风险提示，例如 Spring Security 的认证授权边界、MyBatis SQL 注入风险、前后端接口字段一致性等问题等[12]。

考拉在线考试系统采用 Spring Boot + Vue + MySQL 的前后端分离架构，且系统中包含题库、试卷、考试记录、错题、消息和日志等较多数据库实体。项目早期已有“数据库重构”“重绘 ER 图并进行集体评审”的实践。AI 可以嵌入这一环节，辅助检查数据库设计是否支持多条件查询、成绩统计、错题关联、试卷题目关系维护等功能。例如，AI 可以根据“学生—试卷—答题记录—错题本—题目—学科”的关系，提示是否需要中间表、是否存在一对多关系表达不清、是否缺少创建时间和状态字段等。

2.4 编码实现阶段：AI 代码生成、代码解释、代码审查与重构建议

编码实现阶段是当前 AI 工具最成熟的应用场景。GitHub Copilot 等工具已经支持代码补全、自然语言生成函数、解释代码、生成单元测试、重构建议和 Pull Request 代码审查[5][6]。对于本项目而言，AI 可以辅助生成后端 Controller、Service、Mapper 的样板代码，也可以根据接口说明生成前端 Axios 调用、表单校验逻辑和 Element UI 表格组件[5]。对于重复性较强的 CRUD 功能，例如学生管理、教师管理、学科管理、题库管理，AI 可以显著减少模板化代码编写时间。

但是，AI 生成代码也存在风险。在线考试系统涉及用户认证、成绩数据、权限控制和考试记录，不能简单接受 AI 生成的安全相关代码。AI 可能遗漏鉴权判断、错误处理、事务边界、SQL 参数绑定和输入校验；已有研究也提示，AI 代码审查工具在安全漏洞识别方面仍不能替代专门安全测试和人工审计[10]。因此，编码阶段的 AI 匹配场景更适合选择“样板代码生成”和“代码审查辅助”，并保留人工审查、安全检查和运行测试作为最终质量门禁。

2.5 测试验证阶段：AI 生成测试用例、边界场景与缺陷预测

测试验证阶段是本项目中最适合做原型验证的 AI 场景。现有软件过程中的验证方式主要是手动回归测试、SQL 完整性验证和压力模拟，测试用例依赖人工经验编写。考拉在线考试系统的业务功能具有明确输入、输出和状态变化，例如登录、试卷查询、题目增删改查、发布任务试卷、参加考试、提交答案、查看成绩、错题收录等，适合 AI 根据需求文档自动生成测试用例[8]。

AI 在测试阶段可以完成以下工作：

1. 根据需求说明书生成正常流程、异常流程、边界值和权限类测试用例；
2. 根据数据库表结构生成 SQL 完整性测试场景，如空值、重复记录、外键约束和多条件查询；
3. 根据接口说明生成 Postman 测试请求模板；
4. 根据前端页面功能生成手动测试清单；
5. 根据缺陷记录推测高风险模块，建议优先回归测试范围。

对本项目而言，最典型的验证场景是“多条件查询测试”“试卷发布与考试提交测试”“自动判分与错题收录测试”。这些场景既与项目功能紧密相关，又容易量化比较人工方式和 AI 辅助方式在用例数量、覆盖点、异常场景数量和编写耗时上的差异。因此，本报告建议后续原型验证优先选择“AI 辅助生成测试用例”。

2.6 运维监控阶段：AI 异常检测、日志分析与根因定位

虽然前期软件过程剪裁中将“运行与维护过程”裁剪掉，原因是课程项目主要用于现场演示，不涉及长期上线部署和用户热线支持，但考拉在线考试系统本身设计了用户日志、系统数据概览、成绩统计和行为监控功能。如果系统未来扩展为真实在线平台，运维监控阶段将具备 AI 引入价值。

AI 可以分析用户登录日志、考试提交日志、接口错误日志和数据库慢查询日志，辅助识别异常行为。例如，多名学生同时提交试卷导致响应变慢时，AI 可以根据日志摘要提示可能的数据库连接池、事务锁或接口并发问题；某个用户频繁登录失败时，AI 可以提示是否存在账号安全风险；某类题目提交后频繁出现判分异常时，AI 可以提示检查自动判分规则或题目答案格式。对于本课程项目，运维监控 AI 场景不作为第一优先级，但可以作为“未来扩展场景”纳入改进后过程定义。

3. 现有软件过程与项目特点概述

3.1 现有软件过程

前期《软件过程剪裁与定义文档》基于 ISO/IEC 12207 标准框架，并结合“数据库驱动、快速交付、敏捷协作”的课程项目特点，对完整软件过程进行了剪裁。当前保留的核心过程主要包括：

过程类别	过程名称	当前主要活动	当前输出物
主要过程	需求定义过程	痛点收集、情境模拟、快速裁减、需求文档整理	需求规格说明书、用例图
主要过程	设计定义过程	SpringBoot+Vue 架构选型、ER 图重绘、数据库评审、Swagger 接口约定	规范化 ER 图、数据库脚本、接口规范
主要过程	实现与集成过程	4 人分头开发、Gitee/GitHub 推送、互助 Debug、前后端联调	后端 API、前端页面、集成软件
主要过程	验证过程	手动回归测试、SQL 完整性验证、压力模拟	测试用例表、缺陷记录
支持过程	配置管理过程	版本控制、文档传递、基线锁定	Git 记录、基线文档
管理过程	项目监控与控制	每周腾讯会议、微信群同步、动态人力调配	会议记录、任务看板

现有过程具有明显的学生团队特征：团队规模小、周期短、正式 QA 岗位缺失、需求和测试文档主要依赖人工编写，项目监控主要依靠微信群和会议同步。

3.2 项目功能特点

考拉在线考试系统的主要功能包括：

- 1. 管理员端：身份验证登录、系统数据概览、学生账户管理、教师账户管理、学科管理、试卷管理、题库资源管理、题目信息创编、任务试卷发布、成绩与作答情况查询、消息发送、用户日志查看、个人资料修改等。
- 2. 学生端：用户注册与登录、查看并完成老师发布的任务、查看待做试卷、参加考试、查看考试记录、查看个人错题集、修改个人资料、接收教师消息、查看个人账户活动动态等。
- 3. 系统创新点：快速高效的 用户查询、智能错题收录与分析、动态题库管理、灵活试卷生成机制、互动式师生通讯、实时成绩分析反馈、行为日志监控和动态数据分析。

这些功能决定了本项目具备较多适合 AI 介入的场景：一方面，系统中存在大量结构化对象，如用户、题目、试卷、学科、成绩、消息、日志，适合 AI 辅助分析和生成；另一方面，项目开发过程中需求、接口、测试用例、SQL 查询、异常场景等都具有较强的重复性和规则性，适合使用大语言模型、代码助手、AI 测试工具和智能日志分析方法进行过程优化。

4. 重点匹配场景分析

4.1 场景一：AI 辅助测试用例生成与边界场景补全

对应过程：验证过程。

现有验证过程主要依靠人工手动回归测试、SQL 完整性验证和压力模拟。该过程能够覆盖核心功能，但容易受测试人员经验影响，尤其容易遗漏异常流程、边界值、权限控制和数据一致性问题。考拉在线考试系统涉及大量规则性业务，如登录验证、试卷发布、考

试提交、自动判分、错题收录、题库筛选和成绩查询，均适合转化为测试用例。

AI 可根据软件需求说明书中的功能描述自动生成测试用例表，包括用例编号、测试目标、前置条件、输入数据、执行步骤、预期结果和优先级。例如，对于“管理员创编并发布任务试卷”，AI 可以补充以下测试场景：未选择题目时发布、试卷时间为空、考试时长为负数、学生重复提交、考试结束后提交、题目被删除后历史试卷是否可查看等。对于“多条件题库查询”，AI 可以补充关键词为空、学科不存在、题型与年级组合无结果、分页边界、模糊搜索特殊字符等场景。

该场景的优势是可验证性强，后续可以直接比较人工测试用例和 AI 生成测试用例在数量、覆盖维度、异常场景数量、耗时等方面的差异，因此建议作为第三个交付文档《AI 技术应用验证报告》的首选原型验证场景。

4.2 场景二：AI 辅助需求条目整理与一致性检查

对应过程：需求定义过程。

现有需求定义过程包括痛点收集、情境模拟和快速裁减，强调通过社交媒体调研、角色扮演和微信群投票确定项目范围。这种方式适合小团队快速达成共识，但容易出现需求描述粒度不一致、功能边界不清、异常流程遗漏等问题。例如，需求文档中管理员既承担教师管理、题库管理、试卷管理，又承担消息发送和日志查看职责，AI 可以帮助检查角色权限是否过大、功能描述是否重复、学生端和管理员端是否存在交互断点。

AI 在该场景中的作用包括：将自然语言需求整理为标准需求条目；为每个功能生成用户故事和验收标准；检查需求之间是否存在冲突；识别缺少异常流程的功能；将需求与测试用例建立追踪关系。对于本项目，AI 可以重点检查“考试提交—成绩生成—错题收录—成绩查询”这一链路是否在需求中完整闭环，并提示是否需要补充网络异常、重复提交、答案为空、简答题人工判分等边界情况。

该场景风险较低，因为 AI 输出主要是文档建议，不会直接改变代码和数据；同时它能为后续设计、编码和测试提供更清晰的输入，因此适合在改进方案中作为需求阶段的关键 AI 节点。

4.3 场景三：AI 辅助 SQL 检查、缺陷定位与 Debug

对应过程：实现与集成过程、验证过程。

现有实现与集成过程采用分头开发、每日推送、互助 Debug 和定期联调。团队曾通过屏幕共享共同解决后端报错或跨域问题，也在验证阶段手动运行核心 SQL 检查结果与数据库记录是否一致。该过程虽然直接有效，但高度依赖成员经验，遇到复杂 SQL、多表连接、权限过滤或分页排序问题时，排查成本较高。

AI 可以作为 Debug 助手，读取报错信息、SQL 语句、接口返回和部分代码片段，推测可能原因并给出排查步骤。例如，针对多条件查询没有返回预期结果的问题，AI 可以提示检查 where 条件是否拼接错误、字符串模糊匹配是否使用正确、前端参数名是否与后端接收字段一致、分页偏移量是否计算错误、数据库字段类型是否不匹配等。针对 Spring Boot 接口报错，AI 可以辅助解释异常栈，并建议检查 Controller 映射、参数注解、Service 注入、MyBatis Mapper XML 和数据库连接配置。

该场景特别适合本项目，因为系统涉及大量 CRUD、筛选查询、成绩统计和数据关联操作。但 AI 给出的修复建议必须经过人工验证，尤其不能直接接受可能影响数据安全或权限判断的代码修改。

5. 不优先选择的场景说明

5.1 不优先选择“全自动代码生成”

虽然 AI 代码生成可以明显提升开发速度，但本项目涉及登录认证、权限控制、考试数据和成绩记录，若直接使用 AI 自动生成大量核心代码，可能引入安全漏洞、业务规则偏差或难以维护的代码。因此，本报告不建议将“全自动代码生成”作为主要改进方向，而是将其限定为 CRUD 样板代码、注释生成、代码解释和局部重构建议。

5.2 不优先选择“AI 自动替代需求评审”

AI 可以发现需求表述问题，但不能替代用户和开发团队的范围决策。考拉在线考试系统曾通过微信群投票裁剪“防作弊监控”等功能，这类取舍受课程时间、团队能力和项目目标影响，不能只由 AI 判断。因此需求阶段应采用“AI 提建议，团队做决策”的方式。

6. 最终推荐的 5 个 AI 应用场景

结合调研结果、项目实际过程和可落地性，本报告最终推荐以下 5 个 AI 应用场景进入后续改进方案设计：

推荐优先级	AI 应用场景	对应软件过程	推荐理由	后续验证可能性
1	AI 辅助测试用例生成与边界场景补全	验证过程	与现有人工测试痛点高度匹配，容易形成对比数据	很高，建议作为原型验证场景
2	AI 辅助需求条目整理与一致性检查	需求定义过程	可提升 SRS 完整性和可测试性，风险较低	较高，可比较需求遗漏数量
3	AI 辅助 SQL 检查、缺陷定位与 Debug	实现与集成、验证过程	贴合多表查询、接口联调和错误排查需求	较高，可记录 Debug 耗时差异

其中，**AI 辅助测试用例生成与边界场景补全**最适合作为第三阶段“AI 技术应用验证报告”的验证对象。原因是该场景输入清晰、输出可量化、风险可控，且与系统中的题库查询、考试提交、自动判分、错题收录等核心功能直接相关。通过对比人工测试用例和 AI 生成测试用例，可以较直观地展示 AI 对软件过程效率和质量的提升。

7. 场景匹配结论

通过对 2024—2026 年 AI 软件工程应用趋势的调研可以看出，AI 正在从代码补全工具发展为贯穿需求、设计、编码、测试和运维的协作型工程工具。但 AI 的价值并不在于替代软件过程，而在于嵌入已有过程，承担候选生成、遗漏检查、重复任务自动化和辅助分析工作。

对于考拉在线考试系统而言，现有软件过程具有小团队、短周期、数据库驱动、快速交付、人工测试为主的特点。因此，最适合引入 AI 的不是大型自动化平台，而是轻量、可复核、能直接嵌入现有活动的 AI 辅助节点。结合项目功能和过程痛点，本报告推荐将 AI 重点引入测试用例生成、需求一致性检查、数据库与接口设计评审、SQL/缺陷定位、项目过程记录五个场景。

后续《软件过程改进方案设计文档》应围绕上述场景进一步定义 AI 参与节点、角色职责、工具链、质量标准 and 风险控制机制；《AI 技术应用验证报告》建议选择“AI 辅助测试用例生成”作为最小可行验证场景，使用真实 AI 工具针对考拉在线考试系统的某一核心功能生成测试用例，并与人工测试方式进行效率和质量对比。

参考文献

[1] DORA. Accelerate State of DevOps Report 2024. 2024. <https://dora.dev/research/2024/dora-report/>

[2] DORA. State of AI-assisted Software Development 2025. 2025. <https://dora.dev/dora-report-2025/>

[3] Stack Overflow. 2025 Developer Survey: AI. 2025. <https://survey.stackoverflow.co/2025/ai>

- [4] JetBrains. *The State of Developer Ecosystem Report 2024*. 2024. <https://www.jetbrains.com/lp/devecosystem-2024/>
- [5] GitHub Docs. *GitHub Copilot documentation*. 2026. <https://docs.github.com/copilot>
- [6] GitHub Blog. *Copilot code review now generally available*. 2025. <https://github.blog/changelog/2025-04-04-copilot-code-review-now-generally-available/>
- [7] Cheng, H., Husen, J. H., Lu, Y., Racharak, T., Yoshioka, N., Ubayashi, N., & Washizaki, H. *Generative AI for Requirements Engineering: A Systematic Literature Review*. 2024. <https://arxiv.org/abs/2409.06741>
- [8] Garousi, V., et al. *A Roadmap for Software Testing in Open-Collaborative and AI-Powered Era*. 2024. <https://arxiv.org/html/2406.05438v2>
- [9] Pandey, R., et al. *Transforming Software Development: Evaluating the Efficiency and Challenges of GitHub Copilot in Real-World Projects*. 2024. <https://arxiv.org/abs/2406.17910>
- [10] Amro, A., & Alalfi, M. H. *GitHub's Copilot Code Review: Can AI Spot Security Flaws Before You Commit?* 2025. <https://arxiv.org/abs/2509.13650>
- [11] Zadenoori, M. A., Dąbrowski, J., Alhoshan, W., Zhao, L., & Ferrari, A. *Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review*. 2025. <https://arxiv.org/abs/2509.11446>
- [12] Kooy, S. J., Piest, J. P. S., & Bemthuis, R. H. *Impact and Implications of Generative AI for Enterprise Architects in Agile Environments: A Systematic Literature Review*. 2025. <https://arxiv.org/abs/2510.22003>